CS327: Compilers

Language Specification & Reference Manual

Language Name: ***OPEX***

Version 1.0

Group Members:

Gaurav Budhwani (22110085)

John Twipraham Debbarma (22110108)

Harshit (22110095)

Instructor: Prof. Shouvick Mondal

February 6, 2026

Contents

1 Introduction **3**

1.1 Purpose and Scope . . . 3

1.2 Language Paradigm . . . 3

1.3 Artifact Specification . . . 3

2 Lexical Specification (Assignment A2) **3**

2.1 Keywords . . . 3

2.2 Token Patterns . . . 4

2.3 Operators . . . 5

3 Syntax Specification (Assignment A3) **5**

3.1 Grammar Rules (BNF) . . . 5

4 Sample Artifacts **6**

4.1 Sample Program: Matrix Operations Inference . . . 6

5 Semantic Specification (Assignment A4) **7**

5.1 Type System . . . 7

5.1.1 Primitive Types . . . 8

5.1.2 The `IMAGE` Type . . . 8

5.1.3 The MATRIX Type . . . 8

5.1.4 The KERNEL Type . . . 8

5.1.5 The PIXEL Type . . . 9

5.1.6 Composite Types: ARRAY and STRUCT . . . 9

5.1.7 ARRAY . . . 9

5.1.8 STRUCT . . . 9

5.1.9 Type Inference with AUTO . . . 9

5.1.10 Type Compatibility and Conversions 9

6 Expressions and Operators **10**
6.1 Primary Expressions 10
6.2 Arithmetic Operators 10
6.3 Relational and Logical Operators 10
6.4 Assignment Operators 10
6.5 Pixel Access and Slicing 10
6.6 Convolution Operator (`**`) 11
6.7 Pipeline Operator (`|>`) 11
6.8 Operator Precedence and Associativity Table 11
6.9 Arrays and Slicing Semantics 11

7 Scope and Name Binding **12**

8 Control Flow Semantics **12**
8.1 Conditional Statements 12
8.2 Looping Constructs 12

9 Input / Output **13**
9.1 PRINT and INPUT Statements 13
9.2 String Interpolation 13
9.3 File I/O for Images 13

10 Error Handling **13**

11 Design Decisions and Limitations **13**

A Implementation Artifacts **15**
A.1 Lexer Source (lexer.l) 15
A.2 Build Configuration (Makefile) 16

B Sample Programs **17**
B.1 Edge Detection Pipeline 17
B.2 Prime Detection 17
B.3 Prime Factorisation 18
B.4 Fibonacci Sequence (Recursive) 19
B.5 Binary Search 19
B.6 GCD (Recursive) 20
B.7 Bubble Sort 21
B.8 Palindrome Checker 21
B.9 Graph DFS (Adjacency Matrix) 22
B.10 Binary Tree In-Order Traversal 23
B.11 Floyd-Warshall Algorithm 24
B.12 Matrix Multiplication 25
B.13 Stack Implementation 26
B.14 Queue Implementation (Circular Buffer) 27

1 Introduction

1.1 Purpose and Scope

This document serves as the formal Language Reference Manual (LRM) for **OPEX**, a modern, statically-typed programming language designed for high-performance data processing and algorithmic clarity.

While standard educational languages (like C-subsets) rely on verbose syntax and unsafe memory practices, OPEX introduces modern paradigms such as **Type Inference**, **Matrix Slicing**, and **First-Class Strings**. This specification defines the rules for Lexical (A2), Syntax (A3), and Semantic (A4) analysis. OPEX is focused especially for Computer Vision (CV) tasks.

1.2 Language Paradigm

OPEX is designed to be "Better than C" for scientific applications:

- **Type Inference:** The `AUTO` keyword allows the compiler to deduce types, reducing boilerplate code.
- **Vectorized Operations:** Arithmetic operators (`+, -`) are overloaded for Arrays/-Matrices, allowing direct mathematical notation (e.g., `MatC := MatA + MatB`).
- **Safety & Clarity:** Explicit block terminators (`ENDIF`, `ENDFOR`) eliminate dangling-else ambiguity, while `FOREACH` loops prevent common off-by-one errors.

1.3 Artifact Specification

This document includes the artifacts for the **Lexical Analyzer (A2)**:

1. **Source Specifications:** The `lexer.l` file (Flex).
2. **Build System:** A `Makefile` for automated building.
3. **Test Suite:** Comprehensive test cases for the advanced features.

2 Lexical Specification (Assignment A2)

2.1 Keywords

OPEX reserves the following case-sensitive keywords:

Category	Keywords
Program Structure	PROGRAM, BEGIN, END, FUNCTION, RETURN, MAIN
Data Types and Declarations	VAR, AUTO, INT, FLOAT, BOOL, STRING, VOID, ARRAY, OF
CV / Domain Types	IMAGE, MATRIX, KERNEL, PIXEL, ARRAY, OF
Composite Types	STRUCT, ENDSTRUCT
Control Flow	IF, THEN, ELSE, ENDIF, MATCH, WHEN, ENDMATCH
Loops	WHILE, DO, ENDWHILE, FOR, FOREACH, IN, TO, STEP, ENDFOR
I/O	PRINT, INPUT
Image Operations	LOAD, SAVE, GRAYSCALE, HSV, BLUR, THRESHOLD, CANNY, RESIZE, ROTATE, ERODE, DILATE, FLIP
Logic	TRUE, FALSE, AND, OR, NOT, NULL

2.2 Token Patterns

- **Identifier:** `[a-zA-Z_][a-zA-Z0-9_]*`
- **Integer Literal:** `[0-9]+`
- **Hexadecimal Literal:** `0[xX][0-9a-fA-F]+`
- **Float Literal:** `[0-9]+\.[0-9]*([eE][+-]?[0-9]+)?`
- **String Literal:** `"[^"\n]*"`
- **Range Operator:** `\.\.`
- **Exponentiation Operator:** `\*\*`
- **Pipe Arrow Operator:** `\|>`
- **Assignment Operator:** `:=`
- **Plus Assignment Operator:** `\+=`
- **Relational Operators:** `==` `!=` `<` `>` `<=` `>=`
- **Parentheses:** `( )`
- **Brackets:** `[ ]`
- **Separators:** `,` `:` `;`
- **Comments:** `//.*` (Single Line) and `/* ... */` (Multi-line)

2.3 Operators

- **Arithmetic:** `+`, `-`, `*`, `/`, `%` (Modulo), `++`, `--`
- **Slice/Range:** `..` (e.g., `0..10`)
- **Assignment:** `:=`
- **Relational:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Logical:** `AND`, `OR`, `NOT`

3 Syntax Specification (Assignment A3)

The Context-Free Grammar (CFG) supports the advanced feature set, including global function definitions.

3.1 Grammar Rules (BNF)

Note: The Start Symbol is ***Program****.*

```
1.  Program         -> PROGRAM Identifier GlobalDecls FuncList MAIN Block END

2.  GlobalDecls     -> GlobalDecl GlobalDecls | epsilon
3.  GlobalDecl      -> STRUCT Identifier StructBlock ENDSTRUCT
                     | VAR Identifier : Type ;

4.  FuncList        -> Function FuncList | epsilon
5.  Function        -> FUNCTION Identifier ( ParamList ) : Type BEGIN Block END
                     | FUNCTION Identifier ( ParamList ) : VOID BEGIN Block END

6.  ParamList       -> ParamDecl , ParamList | ParamDecl | epsilon
7.  ParamDecl       -> Identifier : Type

8.  Block           -> DeclList StatementList
9.  DeclList        -> Decl DeclList | epsilon
10. Decl            -> VarDecl | StructDecl
11. VarDecl         -> VAR Identifier : Type ;
                     | AUTO Identifier := Expression ; /* Type Inference */
12. StructDecl      -> STRUCT Identifier StructBlock ENDSTRUCT
13. StructBlock     -> VarDecl StructBlock | VarDecl

14. Type            -> BaseType | ArrayType | Identifier
15. BaseType        -> INT | FLOAT | BOOL | STRING | VOID
16. ArrayType       -> ARRAY [ Number ] OF Type
                     | ARRAY [ Number ][ Number ] OF Type

17. StatementList   -> Statement StatementList | epsilon
18. Statement       -> Assignment ;
                     | IfStatement
```

```
                    | WhileStatement
                    | ForStatement
                    | ForeachStatement
                    | IOStatement ;
                    | ReturnStmt ;

19. Assignment     -> LValue := Expression
                    | LValue ++
                    | LValue --
20. ReturnStmt     -> RETURN Expression | RETURN

21. IfStatement    -> IF Condition THEN StatementList ELSE StatementList ENDIF
                    | IF Condition THEN StatementList ENDIF

22. WhileStatement -> WHILE Condition DO StatementList ENDWHILE
23. ForStatement   -> FOR Identifier := Expr TO Expr STEP Expr DO StatementList
ENDFOR
                    | FOR Identifier := Expr TO Expr DO StatementList ENDFOR
24. ForeachStatement -> FOREACH Identifier IN Identifier DO StatementList ENDFOR

25. LValue         -> Identifier | Identifier Accessor
26. Accessor       -> [ Expression ] Accessor
                    | [ Expression .. Expression ] Accessor /* Slicing */
                    | . Identifier Accessor
                    | epsilon

27. Expression     -> SimpleExpr RelOp SimpleExpr | SimpleExpr
28. SimpleExpr     -> Term AddOp SimpleExpr | Term
29. Term           -> Factor MultOp Term | Factor
30. Factor         -> LValue | Number | StringLit | ( Expression )
                    | FunctionCall | TRUE | FALSE
31. FunctionCall   -> Identifier ( ArgList )
32. ArgList        -> Expression , ArgList | Expression | epsilon
33. KernelDecl    -> KERNEL IDENTIFIER := KernelLit
34. KernelLit     -> [ KernelRow KernelRows ]
35. KernelRows    -> , KernelRow KernelRows | epsilon
36. KernelRow     -> [ NumList ]
37. PipeExpr      -> OrExpr |> PipeExpr | OrExpr
38. ConvExpr      -> UnaryExpr ** ConvExpr | UnaryExpr
```

4 Sample Artifacts

4.1 Sample Program: Matrix Operations Inference

This demonstrates **Type Inference**, **Matrix Slicing**, and **Foreach Loops**.

```
PROGRAM MatrixMagic
    // Global Struct Definition
    STRUCT Pixel
```

```
        r : INT
        g : INT
        b : INT
    ENDSTRUCT

    FUNCTION initMatrix() : VOID
    BEGIN
        PRINT "Initializing...";
    END

MAIN
BEGIN
    // Type Inference: Compiler knows 'rows' is INT
    AUTO rows := 10;

    // 2D Array Declaration
    VAR matrixA : ARRAY[10][10] OF INT;
    VAR matrixB : ARRAY[10][10] OF INT;
    VAR sumMatrix : ARRAY[10][10] OF INT;

    // FOREACH Loop (Iterates through rows)
    FOREACH row IN matrixA DO
        FOREACH val IN row DO
            val := 5;
        ENDFOR
    ENDFOR

    // Vectorized Addition (Conceptual for Semantics)
    // In A4, this generates loops automatically
    sumMatrix := matrixA + matrixB;

    // Matrix Slicing (Accessing a sub-region)
    // Extracts rows 0 to 5
    VAR subRegion : ARRAY[5][10] OF INT;
    subRegion := matrixA[0..5];

    PRINT "Matrix Processing Complete";
END
```

Listing 1: Modern Features in OPEX

5 Semantic Specification (Assignment A4)

5.1 Type System

OPEX is a statically typed language. Every variable has a well-defined type that is determined at compile time.

- Variables declared using `VAR` must explicitly specify a type.
- Variables declared using `AUTO` infer their type from the initializing expression.
- Once inferred, the type of an `AUTO` variable cannot be changed.

OPEX does not allow implicit type coercion. Any operation involving incompatible types results in a compile-time semantic error.

5.1.1 Primitive Types

Type	Size	Description	Default
`INT`	32-bit signed	Integers, pixel coordinates, counters	`0`
`FLOAT`	64-bit IEEE 754	Floating-point, kernel weights, normalized pixels	`0.0`
`BOOL`	8-bit	Logical values	`FALSE`
`STRING`	Dynamic	UTF-8 character sequences with interpolation	`""`

5.1.2 The `IMAGE` Type

`IMAGE` is a first-class type representing a 2D image with one or more channels. An `IMAGE` value carries its width, height, and channel count as part of its type metadata at runtime. Pixel values are stored as unsigned 8-bit integers (0–255) per channel by default, or as `FLOAT` for normalized images.

```
VAR img     : IMAGE                 // Uninitialized, set via LOAD
VAR canvas  : IMAGE(640, 480, 3)    // 640x480, 3-channel (RGB)
VAR mask    : IMAGE(100, 100, 1)    // 100x100, single-channel (
   grayscale)
```

Properties are accessible via dot notation: `img.width`, `img.height`, `img.channels`.

Channel accessors include `img.red`, `img.green`, `img.blue` (each returning a single-channel `IMAGE`), and `img.gray`, which is a shortcut for grayscale conversion.

5.1.3 The MATRIX Type

`MATRIX` represents a 2D array of `FLOAT` values, used for homography matrices, transformation matrices, and general linear algebra. Unlike `IMAGE`, `MATRIX` values have no channel dimension and no 0–255 clamping.

```
VAR transform : MATRIX(3, 3)        // 3x3 transformation matrix
VAR data      : MATRIX(100, 50)     // 100-row, 50-column data matrix
```

5.1.4 The KERNEL Type

`KERNEL` is a small, immutable `MATRIX` specifically for convolution operations. Kernels must have odd dimensions (1×1, 3×3, 5×5, 7×7, ...) so that they have a well-defined center element. The compiler enforces this constraint at compile time for literal kernels and at runtime for dynamically constructed ones.

```
// Inline kernel literal
KERNEL gaussianBlur := [[ 1,  2,  1],
                        [ 2,  4,  2],
                        [ 1,  2,  1]]

// The compiler normalizes the kernel (divides by sum of weights)
// unless the user explicitly opts out with KERNEL RAW
```

5.1.5 The PIXEL Type

`PIXEL` is a compound type representing the channel values at a single (x, y) coordinate. For a 3-channel image, a `PIXEL` has fields `.r`, `.g`, `.b`. For a single-channel image, a `PIXEL` has field `.v` (value). `PIXEL` is primarily used as the return type of the subscript operator on `IMAGE`.

```
VAR px : PIXEL := img[100, 200]    // Access pixel at (x=100, y=200)
PRINT "Red: {px.r}, Green: {px.g}, Blue: {px.b}"
```

5.1.6 Composite Types: ARRAY and STRUCT

5.1.7 ARRAY

`ARRAY` is a fixed-size, homogeneous collection. One-dimensional and two-dimensional arrays are supported.

```
VAR histogram  : ARRAY[256] OF INT          // 1D: 256-bin histogram
VAR grid       : ARRAY[10][10] OF FLOAT     // 2D: 10x10 grid
VAR kernelBank : ARRAY[8] OF KERNEL         // Array of kernels
```

5.1.8 STRUCT

`STRUCT` defines a named product type (record) with named fields. Structs are terminated with `ENDSTRUCT`.

```
STRUCT BoundingBox
    VAR x      : INT
    VAR y      : INT
    VAR width  : INT
    VAR height : INT
    VAR label  : STRING
    VAR conf   : FLOAT
ENDSTRUCT
```

5.1.9 Type Inference with AUTO

The `AUTO` keyword instructs the compiler to deduce the variable's type from its initializer expression. `AUTO` declarations must always include an initializer. The inferred type is fixed at the declaration point and cannot change.

```
AUTO count := 0              // Inferred as INT
AUTO ratio := 3.14           // Inferred as FLOAT
AUTO img   := LOAD("f.png")  // Inferred as IMAGE
AUTO gray  := GRAYSCALE(img) // Inferred as IMAGE (1-channel)
```

5.1.10 Type Compatibility and Conversions

OPEX performs **no implicit type conversions**. An `INT` cannot be silently promoted to `FLOAT`; this must be done explicitly using the built-in `FLOAT(expr)` and `INT(expr)` cast expressions. This prevents the subtle precision-loss bugs that plague C vision code (for example, integer division in gradient calculations).

`IMAGE` and `MATRIX` are not interchangeable. An explicit conversion function `TO_MATRIX(img)` extracts a single-channel `IMAGE` into a `MATRIX` of `FLOAT` values (0.0 to 255.0), and `TO_IMAGE(mat)` performs the reverse with clamping.

6 Expressions and Operators

6.1 Primary Expressions

A primary expression is an identifier, a literal (integer, float, string, boolean, kernel), a parenthesized expression, or a function call. Pixel access (`img[x, y]`) and field access (`obj.field`) are also primary expressions.

6.2 Arithmetic Operators

The arithmetic operators `+`, `-`, `*`, `/`, and `%` operate on `INT` and `FLOAT` values. When applied to `IMAGE` operands, `+` and `-` perform element-wise (pixel-wise) addition and subtraction with saturation arithmetic (clamped to 0–255). This allows direct notation like `result := imgA + imgB`.

Scalar–`IMAGE` operations are also supported: `img * 2` multiplies every pixel value by 2 (with clamping). Similarly, `img + 50` brightens the image by adding 50 to each channel.

6.3 Relational and Logical Operators

Relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) produce `BOOL` results. Logical operators `AND`, `OR`, `NOT` operate on `BOOL` values and short-circuit as expected.

6.4 Assignment Operators

The primary assignment operator is `:=`. Compound assignments `+=`, `-=`, `*=`, `/=` are provided for convenience. The increment and decrement operators `++` and `--` are postfix only and apply only to `INT` variables.

6.5 Pixel Access and Slicing

The subscript operator on `IMAGE` supports several access patterns. These are among the most distinctive features of OPEX.

```
// Single pixel access (returns PIXEL)
VAR px : PIXEL := img[x, y]

// Row slice (returns 1-row IMAGE)
VAR row : IMAGE := img[*, y]

// Column slice (returns 1-column IMAGE)
VAR col : IMAGE := img[x, *]

// Region of Interest / ROI (returns sub-IMAGE)
VAR roi : IMAGE := img[x1..x2, y1..y2]

// Single channel extraction
VAR redChannel : IMAGE := img.red
```

The range operator `..` defines inclusive start and exclusive end, consistent with standard practice. Out-of-bounds access produces a runtime error.

6.6 Convolution Operator (**)

The binary operator ** performs 2D convolution of an IMAGE (left operand) with a KERNEL (right operand), returning a new IMAGE. The compiler verifies that the left operand is an IMAGE (or single-channel IMAGE) and the right operand is a KERNEL.

```
AUTO edges   := grayImg ** sobelX
AUTO blurred := img ** gaussianBlur
```

For multi-channel images, convolution is applied independently to each channel.

6.7 Pipeline Operator (|>)

The pipeline operator |> passes the result of the left expression as the first argument to the function on the right. This enables the left-to-right chaining of image processing stages.

```
// Without pipeline (deeply nested, hard to read):
AUTO result := THRESHOLD(BLUR(GRAYSCALE(LOAD("img.png")), 5), 128)

// With pipeline (reads like a processing flowchart):
AUTO result := LOAD("img.png")
               |> GRAYSCALE
               |> BLUR(5)
               |> THRESHOLD(128)
```

Formally, expr |> func(args...) desugars to func(expr, args...). When the function takes only one argument, the parentheses may be omitted: img |> GRAYSCALE.

6.8 Operator Precedence and Associativity Table

The following table lists operators from highest precedence (top) to lowest (bottom). Operators on the same row have equal precedence.

Prec.	Operator(s)	Assoc.	Description
1	() [] . ++ --	Left	Grouping, access, postfix
2	NOT - (unary)	Right	Logical/arithmetic negation
3	**	Left	Convolution
4	* / %	Left	Multiply, divide, modulo
5	+ -	Left	Addition, subtraction
6	..	Non-assoc	Range (slicing)
7	< > <= >=	Non-assoc	Relational comparison
8	== !=	Non-assoc	Equality comparison
9	AND	Left	Logical AND
10	OR	Left	Logical OR
11	|>	Left	Pipeline
12	:= += -= *= /=	Right	Assignment

6.9 Arrays and Slicing Semantics

Arrays in OPEX are statically sized and indexed starting from zero.

- Array bounds must be compile-time constants.

- Accessing an array outside its declared bounds is a semantic error.
- Slicing using the range operator (`..`) produces a sub-array whose element type matches the original array.

7 Scope and Name Binding

OPEX uses lexical (static) scoping.

- Variables declared within a block are visible only inside that block.
- Inner scopes may shadow identifiers declared in outer scopes.
- Loop control variables introduced by `FOR` and `FOREACH` constructs are local to the loop body.

All identifiers must be declared before use. Use of undeclared identifiers results in a compile-time error.

8 Control Flow Semantics

8.1 Conditional Statements

The condition expression in an `IF` statement must evaluate to a boolean value.

- If the condition evaluates to `TRUE`, the `THEN` block is executed.
- Otherwise, the optional `ELSE` block is executed.

8.2 Looping Constructs

OPEX supports `WHILE`, `FOR`, and `FOREACH` loops.

- `WHILE` loops repeatedly execute their body while the condition remains true.
- `FOR` loops execute over a numeric range with an optional step.
- `FOREACH` loops iterate over the elements of an array in row-major order.

9 Input / Output

9.1 PRINT and INPUT Statements

```
PRINT "Hello, OPEX!"                          // String output
PRINT "Image size: {img.width}x{img.height}" // Interpolation
PRINT value                                    // Any printable value

VAR name : STRING
INPUT name                                     // Read a line from stdin

VAR threshold : INT
INPUT threshold                                // Read and parse an
   integer
```

9.2 String Interpolation

Any string literal may contain {`expression`} blocks that are evaluated at runtime and converted to their string representation. The expression inside braces follows the same rules as any OPEX expression.

```
PRINT "Pixel at ({x},{y}) = ({px.r}, {px.g}, {px.b})"
PRINT "Processing took {elapsed / 1000.0}s"
```

9.3 File I/O for Images

Image file I/O is handled by the built-in `LOAD` and `SAVE` functions. The supported formats are PNG (lossless, with alpha), JPEG (lossy, no alpha), and BMP (uncompressed). The format is determined by the file extension in the path string.

10 Error Handling

OPEX detects and reports errors during all compilation phases.

- Lexical errors include invalid tokens and malformed literals.
- Syntax errors include unexpected tokens or malformed statements.
- Semantic errors include undeclared identifiers, type mismatches, and invalid array accesses.

Upon encountering an unrecoverable error, compilation terminates after reporting the error with an appropriate diagnostic message.

11 Design Decisions and Limitations

OPEX is intentionally designed as a simplified language for educational compiler construction.

- Preprocessor directives and macros are not supported.

- Pointer types and manual memory management are excluded.
- Typedef-based name resolution is omitted to avoid lexical ambiguity.

These design decisions ensure deterministic parsing and simplify semantic analysis using standard compiler construction tools.

A Implementation Artifacts

A.1 Lexer Source (lexer.l)

```
%option noyywrap case-sensitive
%{
#include <stdio.h>
/* In A3, we will include "y.tab.h" here */
int line_num = 1;
%}

/* DEFINITIONS */
DIGIT       [0-9]
LETTER      [a-zA-Z]
ID          {LETTER}({LETTER}|{DIGIT}|_)*
NUMBER      {DIGIT}+(\.{DIGIT}+)?
STRING      \"([^\"\\\n]|\\.)*\"

%%
"//".* { /* Ignore Single line */ }
"/*"([^*]|\*+[^*/])*\*+"/"  { /* Ignore Block comment */ }
[ \t\r]                     { /* Ignore whitespace */ }
\n                          { line_num++; }

/* KEYWORDS */
"PROGRAM"|"BEGIN"|"END"     { printf("KW_PROG: %s\n", yytext); }
"FUNCTION"|"RETURN"|"MAIN"  { printf("KW_FUNC: %s\n", yytext); }
"STRUCT"|"ENDSTRUCT"        { printf("KW_COMPOSITE: %s\n", yytext); }
"VAR"|"AUTO"|"ARRAY"|"OF"   { printf("KW_DECL: %s\n", yytext); }
"INT"|"FLOAT"|"BOOL"|"STRING"|"VOID" { printf("TYPE: %s\n", yytext); }
"IMAGE"|"MATRIX"|"KERNEL"|"PIXEL"    { printf("TYPE_CV: %s\n", yytext);
    }
"MATCH"                     { printf("KW_FLOW: %s\n", yytext); }
"WHEN"                      { printf("KW_FLOW: %s\n", yytext); }
"ENDMATCH"                  { printf("KW_FLOW: %s\n", yytext); }
"IF"|"THEN"|"ELSE"|"ENDIF"  { printf("KW_FLOW: %s\n", yytext); }
"FOR"|"FOREACH"|"IN"        { printf("KW_LOOP: %s\n", yytext); }
"TO"|"DO"|"ENDFOR"|"STEP"   { printf("KW_LOOP_CTRL: %s\n", yytext); }
"PRINT"|"INPUT"             { printf("KW_IO: %s\n", yytext); }
"TRUE"|"FALSE"              { printf("BOOL_VAL: %s\n", yytext); }
"AND"|"OR"|"NOT"            { printf("LOGIC_OP: %s\n", yytext); }
"WHILE"                     { printf("KW_LOOP: %s\n", yytext); }
"ENDWHILE"                  { printf("KW_LOOP: %s\n", yytext); }
"TO"                        { printf("KW_LOOP_CTRL: %s\n", yytext); }
"DO"                        { printf("KW_LOOP_CTRL: %s\n", yytext); }
"ENDFOR"                    { printf("KW_LOOP_CTRL: %s\n", yytext); }
"STEP"                      { printf("KW_LOOP_CTRL: %s\n", yytext); }
"LOAD"|"SAVE"               { printf("KW_IMG_IO: %s\n", yytext); }
"GRAYSCALE"|"HSV"           { printf("KW_IMG_OP: %s\n", yytext); }
"BLUR"|"THRESHOLD"|"CANNY"  { printf("KW_IMG_OP: %s\n", yytext); }
"RESIZE"|"ROTATE"           { printf("KW_IMG_OP: %s\n", yytext); }
"ERODE"|"DILATE"|"FLIP"     { printf("KW_IMG_OP: %s\n", yytext); }

/* OPERATORS */
":="        { printf("ASSIGN: %s\n", yytext); }
".."        { printf("RANGE_OP: %s\n", yytext); }
"++"|"--"   { printf("INC_DEC: %s\n", yytext); }
```

```
"+"|"-"      { printf("ADD_OP: %s\n", yytext); }
"*"|"/"|"%"  { printf("MULT_OP: %s\n", yytext); }
"=="|"!="|"<"|">"|"<="|">=" { printf("RELOP: %s\n", yytext); }
"+="                        { printf("PLUS_ASSIGN: %s\n", yytext); }
"-="                        { printf("MINUS_ASSIGN: %s\n", yytext); }
"*="                        { printf("MULT_ASSIGN: %s\n", yytext); }
"/="                        { printf("DIV_ASSIGN: %s\n", yytext); }
".."                        { printf("RANGE_OP: %s\n", yytext); }
"**"                        { printf("CONV_OP: %s\n", yytext); }
"|>"                        { printf("PIPE_OP: %s\n", yytext); }

/* SEPARATORS */
"["|"]"|"("|")"|":"|","|"."|";" { printf("SEP: %s\n", yytext); }

{ID}         { printf("ID: %s\n", yytext); }
{HEXINT}                    { printf("HEX_INT: %s\n", yytext); }
{FLOAT}                     { printf("FLOAT_LIT: %s\n", yytext); }
{INTEGER}                   { printf("INT_LIT: %s\n", yytext); }
{STRING}     { printf("STR: %s\n", yytext); }

.            { printf("ERR: %s line %d\n", yytext, line_num); }
%%

int main() { yylex(); return 0; }
```

Listing 2: Flex Specification for OPEX

A.2 Build Configuration (Makefile)

```
all: lexer

lexer: lex.yy.c
  gcc lex.yy.c -o lexer

lex.yy.c: lexer.l
  flex lexer.l

test: lexer
  ./lexer < tests/sample.axi

clean:
  rm -f lex.yy.c lexer
```

Listing 3: Makefile

B Sample Programs

B.1 Edge Detection Pipeline

This program loads an image, converts it to grayscale, applies Sobel edge detection in both X and Y directions, combines the results, and saves the edge map.

```
PROGRAM EdgeDetection
BEGIN
    // Define Sobel kernels
    KERNEL sobelX := [[-1, 0, 1],
                      [-2, 0, 2],
                      [-1, 0, 1]]

    KERNEL sobelY := [[-1, -2, -1],
                      [ 0,  0,  0],
                      [ 1,  2,  1]]

    // Load and convert to grayscale
    AUTO img  := LOAD("input.jpg")
    AUTO gray := GRAYSCALE(img)

    // Apply Sobel operators
    AUTO edgesX := gray ** sobelX
    AUTO edgesY := gray ** sobelY

    // Combine edge maps (element-wise addition)
    AUTO combined := edgesX + edgesY

    // Threshold to produce binary edge map
    AUTO edgeMap := THRESHOLD(combined, 50)

    SAVE(edgeMap, "edges.png")
    PRINT "Edge detection complete: {img.width}x{img.height}"
END
```

Listing 4: Edge Detection in OPEX

B.2 Prime Detection

This program demonstrates the use of the modulo operator (%), boolean logic, and user input to determine if a number is prime.

```
PROGRAM PrimeCheck
BEGIN
    VAR n : INT;
    VAR i : INT;
    VAR isPrime : BOOL;

    PRINT "Enter a number to check:";
    INPUT n;

    // Assume true initially
    isPrime := TRUE;

    // Handle edge cases
    IF n <= 1 THEN
```

```
        isPrime := FALSE;
    ENDIF

    // Iterate from 2 to n/2
    // OPEX's FOR loop automatically handles the range
    FOR i := 2 TO n / 2 DO
        // If n is divisible by i, it is not prime
        IF (n % i) == 0 THEN
            isPrime := FALSE;
        ENDIF
    ENDFOR

    IF isPrime == TRUE THEN
        PRINT "{n} is a Prime Number";
    ELSE
        PRINT "{n} is NOT a Prime Number";
    ENDIF
END
```

Listing 5: Prime Number Checker in OPEX

B.3 Prime Factorisation

This algorithm uses nested `WHILE` loops to compute the prime factors of a given integer. It demonstrates arithmetic operations and dynamic condition checking.

```
PROGRAM PrimeFactorisation
BEGIN
    VAR n : INT;
    VAR d : INT;

    PRINT "Enter a positive integer:";
    INPUT n;
    PRINT "Prime factors of {n}:";

    // Step 1: Handle divisibility by 2
    WHILE (n % 2) == 0 DO
        PRINT 2;
        n := n / 2;
    ENDWHILE

    // Step 2: Iterate odd numbers starting from 3
    d := 3;
    // Loop while d squared is less than or equal to n
    WHILE (d * d) <= n DO
        // While n is divisible by d, print d and divide n
        WHILE (n % d) == 0 DO
            PRINT d;
            n := n / d;
        ENDWHILE
        // Move to next odd number
        d := d + 2;
    ENDWHILE

    // Step 3: If n is a prime number greater than 2
    IF n > 2 THEN
        PRINT n;
    ENDIF
```

```
END
```

Listing 6: Prime Factorisation Algorithm

B.4 Fibonacci Sequence (Recursive)

Although the current version of OPEX focuses on iteration, this example demonstrates how a global function can be defined to calculate Fibonacci numbers recursively.

```
PROGRAM FibRecursive
    // Global Function Definition
    FUNCTION fib(n : INT) : INT
    BEGIN
        IF n <= 1 THEN
            RETURN n;
        ELSE
            RETURN fib(n - 1) + fib(n - 2);
        ENDIF
    END

MAIN
BEGIN
    VAR terms : INT;
    VAR i : INT;

    PRINT "How many terms?";
    INPUT terms;

    PRINT "Fibonacci Sequence:";
    FOR i := 0 TO terms DO
        PRINT fib(i);
    ENDFOR
END
```

Listing 7: Recursive Fibonacci Function

B.5 Binary Search

This program implements the standard iterative binary search algorithm to find the index of a target value in a sorted array.

```
PROGRAM BinarySearch
BEGIN
    // Initialize a sorted array
    VAR data : ARRAY[10] OF INT;
    VAR target : INT;
    VAR low : INT;
    VAR high : INT;
    VAR mid : INT;
    VAR found : BOOL;

    // Fill array with sorted values
    // data = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
    FOR low := 0 TO 9 DO
        data[low] := low * 2;
    ENDFOR

```

```
    PRINT "Enter value to search for:";
    INPUT target;

    low := 0;
    high := 9;
    found := FALSE;

    WHILE low <= high DO
        mid := (low + high) / 2;

        IF data[mid] == target THEN
            PRINT "Found at index: {mid}";
            found := TRUE;
            // Break loop logic (simulated break)
            low := high + 1;
        ELSE
            IF data[mid] < target THEN
                low := mid + 1;
            ELSE
                high := mid - 1;
            ENDIF
        ENDIF
    ENDWHILE

    IF found == FALSE THEN
        PRINT "Value not found";
    ENDIF
END
```

Listing 8: Iterative Binary Search

B.6 GCD (Recursive)

This program calculates the Greatest Common Divisor of two numbers using the recursive Euclidean algorithm.

```
PROGRAM GCDRecursive
    // Recursive Function Definition
    FUNCTION gcd(a : INT, b : INT) : INT
    BEGIN
        IF b == 0 THEN
            RETURN a;
        ELSE
            RETURN gcd(b, a % b);
        ENDIF
    END

MAIN
BEGIN
    VAR num1 : INT;
    VAR num2 : INT;
    VAR result : INT;

    PRINT "Enter first number:";
    INPUT num1;
    PRINT "Enter second number:";
    INPUT num2;

```

```
    result := gcd(num1, num2);

    PRINT "GCD of {num1} and {num2} is {result}";
END
```

Listing 9: Recursive GCD Function

B.7 Bubble Sort

This program sorts an array of integers in ascending order using the Bubble Sort algorithm.

```
PROGRAM BubbleSort
BEGIN
    VAR arr : ARRAY[5] OF INT;
    VAR i : INT;
    VAR j : INT;
    VAR temp : INT;
    VAR n : INT;

    n := 5;

    // Initialize unsorted array manually for demo
    arr[0] := 64;
    arr[1] := 34;
    arr[2] := 25;
    arr[3] := 12;
    arr[4] := 22;

    PRINT "Sorting array...";

    // Outer loop
    FOR i := 0 TO n - 2 DO
        // Inner loop
        FOR j := 0 TO n - i - 2 DO
            // Swap if current element is greater than next
            IF arr[j] > arr[j + 1] THEN
                temp := arr[j];
                arr[j] := arr[j + 1];
                arr[j + 1] := temp;
            ENDIF
        ENDFOR
    ENDFOR

    PRINT "Sorted Array:";
    FOREACH val IN arr DO
        PRINT val;
    ENDFOR
END
```

Listing 10: Bubble Sort Algorithm

B.8 Palindrome Checker

This program checks if a given integer is a palindrome by reversing it mathematically.

```
PROGRAM PalindromeCheck
```

```
BEGIN
    VAR original : INT;
    VAR reversed : INT;
    VAR remainder : INT;
    VAR temp : INT;

    PRINT "Enter an integer:";
    INPUT original;

    reversed := 0;
    temp := original;

    WHILE temp > 0 DO
        remainder := temp % 10;
        reversed := (reversed * 10) + remainder;
        temp := temp / 10;
    ENDWHILE

    IF original == reversed THEN
        PRINT "{original} is a palindrome.";
    ELSE
        PRINT "{original} is NOT a palindrome.";
    ENDIF
END
```

Listing 11: Numeric Palindrome Check

B.9 Graph DFS (Adjacency Matrix)

This program demonstrates graph traversal using a fixed-size Adjacency Matrix and recursion. It avoids dynamic pointers by using integer indices to represent nodes.

```
PROGRAM GraphDFS
    // Global Adjacency Matrix (5 nodes)
    // 1 indicates connection, 0 indicates no connection
    VAR adj : ARRAY[5][5] OF INT;
    VAR visited : ARRAY[5] OF BOOL;

    FUNCTION dfs(u : INT) : VOID
    BEGIN
        VAR v : INT;

        PRINT "Visiting Node: {u}";
        visited[u] := TRUE;

        // Iterate through all possible neighbors
        FOR v := 0 TO 4 DO
            // If connected and not yet visited
            IF adj[u][v] == 1 AND visited[v] == FALSE THEN
                dfs(v);
            ENDIF
        ENDFOR
    END

MAIN
BEGIN
    VAR i : INT;
    VAR j : INT;
```

```
27
28      // Initialize Graph (Example Structure)
29      // 0 -- 1 -- 2
30      // |    |
31      // 3 -- 4
32
33      // Clear Matrix
34      FOR i := 0 TO 4 DO
35          visited[i] := FALSE;
36          FOR j := 0 TO 4 DO
37              adj[i][j] := 0;
38          ENDFOR
39      ENDFOR
40
41      // Manual Edge Setup (undirected)
42      adj[0][1] := 1; adj[1][0] := 1;
43      adj[0][3] := 1; adj[3][0] := 1;
44      adj[1][2] := 1; adj[2][1] := 1;
45      adj[1][4] := 1; adj[4][1] := 1;
46      adj[3][4] := 1; adj[4][3] := 1;
47
48      PRINT "Starting DFS from Node 0:";
49      dfs(0);
50  END
```

Listing 12: Depth First Search

B.10 Binary Tree In-Order Traversal

This program simulates a Binary Tree using an array, where for any node at index `i`, the left child is at `2*i + 1` and the right child is at `2*i + 2`.

```
1   PROGRAM TreeTraversal
2       // Tree represented as array. -1 indicates NULL.
3       VAR tree : ARRAY[15] OF INT;
4
5       FUNCTION inOrder(index : INT) : VOID
6       BEGIN
7           VAR left : INT;
8           VAR right : INT;
9
10          // Check for "NULL" or out of bounds
11          IF index >= 15 OR tree[index] == -1 THEN
12              RETURN;
13          ENDIF
14
15          left := (2 * index) + 1;
16          right := (2 * index) + 2;
17
18          // Traverse Left
19          inOrder(left);
20
21          // Visit Node
22          PRINT tree[index];
23
24          // Traverse Right
25          inOrder(right);
26      END
```

```

MAIN
BEGIN
    VAR i : INT;

    // Initialize "NULL" values
    FOR i := 0 TO 14 DO
        tree[i] := -1;
    ENDFOR

    // Construct Tree:
    //       10
    //      /  \
    //     5    20
    //    / \
    //   3   7
    tree[0] := 10; // Root
    tree[1] := 5;  // Left of 10
    tree[2] := 20; // Right of 10
    tree[3] := 3;  // Left of 5
    tree[4] := 7;  // Right of 5

    PRINT "In-Order Traversal:";
    inOrder(0);
    // Output should be: 3, 5, 7, 10, 20
END
```

Listing 13: Array-Based Binary Tree

B.11 Floyd-Warshall Algorithm

This algorithm computes the shortest paths between all pairs of nodes in a weighted graph using dynamic programming. It demonstrates complex nested iteration.

```
PROGRAM FloydWarshall
MAIN
BEGIN
    VAR dist : ARRAY[4][4] OF INT;
    VAR i : INT;
    VAR j : INT;
    VAR k : INT;
    VAR INF : INT;

    INF := 9999;

    // Initialize Graph (Weights)
    // INF means no direct edge
    dist[0][0] := 0;   dist[0][1] := 5;   dist[0][2] := INF; dist[0][3]
    := 10;
    dist[1][0] := INF; dist[1][1] := 0;   dist[1][2] := 3;   dist[1][3]
    := INF;
    dist[2][0] := INF; dist[2][1] := INF; dist[2][2] := 0;   dist[2][3]
    := 1;
    dist[3][0] := INF; dist[3][1] := INF; dist[3][2] := INF; dist[3][3]
    := 0;

    // Triple Nested Loop
    FOR k := 0 TO 3 DO
```

```
        FOR i := 0 TO 3 DO
            FOR j := 0 TO 3 DO
                // If path through k is shorter than current path
                IF dist[i][k] + dist[k][j] < dist[i][j] THEN
                    dist[i][j] := dist[i][k] + dist[k][j];
                ENDIF
            ENDFOR
        ENDFOR
    ENDFOR

    PRINT "Shortest Matrix:";
    FOREACH row IN dist DO
        // Only print first element of row for brevity in demo
        PRINT row[0];
    ENDFOR
END
```

Listing 14: All-Pairs Shortest Path

B.12 Matrix Multiplication

This program multiplies two 3×3 matrices. It demonstrates the computational capabilities of OPEX for linear algebra tasks.

```
PROGRAM MatrixMult
BEGIN
    VAR A : ARRAY[3][3] OF INT;
    VAR B : ARRAY[3][3] OF INT;
    VAR C : ARRAY[3][3] OF INT;
    VAR i : INT;
    VAR j : INT;
    VAR k : INT;

    // Initialize Matrix A (Identity)
    FOR i := 0 TO 2 DO
        FOR j := 0 TO 2 DO
            IF i == j THEN
                A[i][j] := 1;
            ELSE
                A[i][j] := 0;
            ENDIF
            // Fill B with some values
            B[i][j] := i + j;
            // Initialize Result C
            C[i][j] := 0;
        ENDFOR
    ENDFOR

    PRINT "Calculating Product...";

    // C[i][j] = Sum(A[i][k] * B[k][j])
    FOR i := 0 TO 2 DO
        FOR j := 0 TO 2 DO
            FOR k := 0 TO 2 DO
                C[i][j] := C[i][j] + (A[i][k] * B[k][j]);
            ENDFOR
        ENDFOR
    ENDFOR
```

```
35
36      PRINT "Result Matrix C (Row 0):";
37      PRINT C[0][0]; PRINT C[0][1]; PRINT C[0][2];
38  END
```

Listing 15: Matrix Multiplication

B.13 Stack Implementation

This program defines a `Stack` structure and simulates methods to push and pop values, demonstrating state management with Structs.

```
1   PROGRAM StackDemo
2       STRUCT Stack
3           data : ARRAY[10] OF INT;
4           top : INT;
5       ENDSTRUCT
6
7       // Global stack instance
8       VAR s : Stack;
9
10      FUNCTION push(val : INT) : VOID
11      BEGIN
12          IF s.top >= 9 THEN
13              PRINT "Error: Stack Overflow";
14          ELSE
15              s.top := s.top + 1;
16              s.data[s.top] := val;
17          ENDIF
18      END
19
20      FUNCTION pop() : INT
21      BEGIN
22          VAR val : INT;
23          IF s.top < 0 THEN
24              PRINT "Error: Stack Underflow";
25              RETURN -1;
26          ELSE
27              val := s.data[s.top];
28              s.top := s.top - 1;
29              RETURN val;
30          ENDIF
31      END
32
33  MAIN
34  BEGIN
35      // Initialize Stack
36      s.top := -1;
37
38      PRINT "Pushing 10, 20, 30...";
39      push(10);
40      push(20);
41      push(30);
42
43      PRINT "Popping values...";
44      PRINT pop(); // Should print 30
45      PRINT pop(); // Should print 20
46      PRINT pop(); // Should print 10
```

```
END
```

Listing 16: Stack Operations

B.14 Queue Implementation (Circular Buffer)

This program implements a fixed-size Circular Queue using an array. It uses the modulo operator to wrap indices around, allowing efficient enqueue and dequeue operations without shifting elements.

```
PROGRAM QueueDemo
    STRUCT Queue
        data : ARRAY[5] OF INT;
        head : INT;
        tail : INT;
        count : INT;
        capacity : INT;
    ENDSTRUCT

    VAR q : Queue;

    FUNCTION enqueue(val : INT) : VOID
    BEGIN
        IF q.count == q.capacity THEN
            PRINT "Error: Queue Full";
        ELSE
            q.data[q.tail] := val;
            // Circular increment: (tail + 1) % capacity
            q.tail := (q.tail + 1) % q.capacity;
            q.count := q.count + 1;
        ENDIF
    END

    FUNCTION dequeue() : INT
    BEGIN
        VAR val : INT;
        IF q.count == 0 THEN
            PRINT "Error: Queue Empty";
            RETURN -1;
        ELSE
            val := q.data[q.head];
            // Circular increment
            q.head := (q.head + 1) % q.capacity;
            q.count := q.count - 1;
            RETURN val;
        ENDIF
    END

MAIN
BEGIN
    // Initialize Queue
    q.head := 0;
    q.tail := 0;
    q.count := 0;
    q.capacity := 5;

    PRINT "Enqueueing 1, 2, 3...";
```

```
48      enqueue(1);
49      enqueue(2);
50      enqueue(3);
51
52      PRINT "Dequeueing...";
53      PRINT dequeue(); // Should be 1
54      PRINT dequeue(); // Should be 2
55
56      PRINT "Enqueueing 4, 5, 6 (Wrapping around)...";
57      enqueue(4);
58      enqueue(5);
59      enqueue(6); // This wraps to index 0 internally
60
61      PRINT "Current Head Value:";
62      PRINT q.data[q.head]; // Should be 3
63  END
```

Listing 17: Circular Queue Operations